

产品介绍

PhyAgentOS：物理智能的“觉醒时刻” --从思考到行动

PhyAgentOS：为模型、仿真环境与异构机器人打造可调度、可检查、可追溯的运行底座，支持即插即用、自动评测、长期记忆与持续进化，让 AI 从 Game 到仿真、再到真机，真正从“会回答”走向“会执行”。

- 项目网站 & demo演示：<https://phy-agent-os.net/>
- 技术报告：<https://phy-agent-os.net/#/tech-report>
- GitHub：<https://github.com/PhyAgentOS/PhyAgentOS>

PhyAgentOS 由中山大学 HCP 实验室、鹏城国家实验室具身智能研究所、X-Era Lab（拓元智慧）联合发起，以 MIT 协议开源。X-Era Lab 作为项目的主要工程贡献者和首批真实场景部署方，为 OS 提供工业级验证反馈和多本体适配经验。

导语：大模型已经很会“想”，为什么机器人还是很难真正用起来？

过去几年，人工智能完成了一次重要转变。

早期的大语言模型更像一位知识丰富的顾问：你提出问题，它给出答案。今天的智能体（Agent）则开始拥有规划、记忆和工具调用能力——它不仅能告诉你“应该怎么做”，还可以搜索资料、编写代码、操作软件，连续完成一整套任务。

但当 Agent 从电脑屏幕走进物理世界，问题突然变得复杂起来。

在数字世界里，点错一个按钮通常可以撤销；在真实世界里，机械臂多移动几厘米，就可能碰倒物体。不同机器人使用不同的传感器、控制接口和动作空间，同一句“把杯子放到托盘里”，在机械臂、移动机器人和仿真平台上，背后可能是三套完全不同的执行系统。

更棘手的是，“**动作执行结束**”并不等于“**任务真的完成**”。

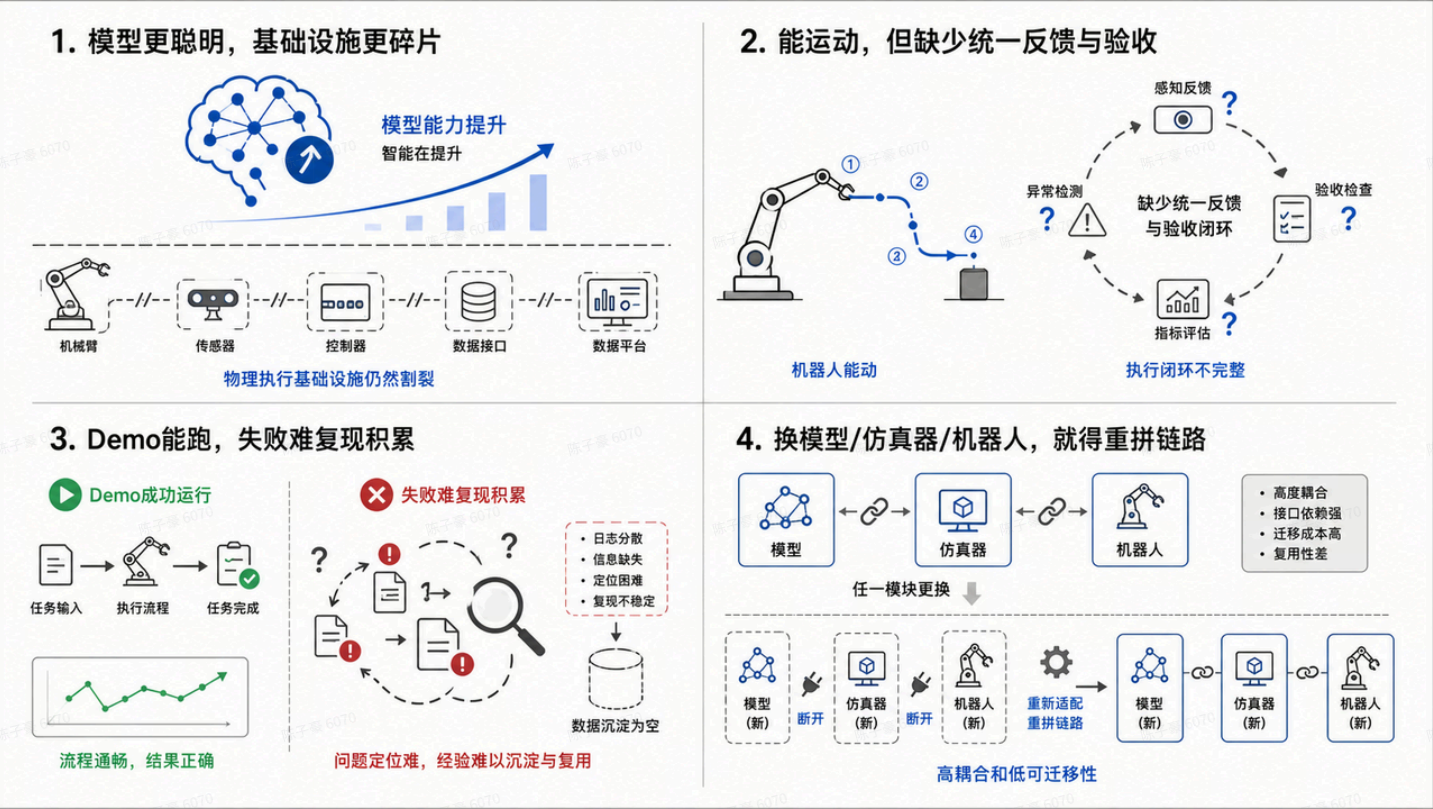
机械臂可能已经走完预设轨迹，杯子却没有被抓住；策略模型可能已经返回成功，目标物体却仍停留在原处。如果系统只知道“命令发出去了”，却不知道“事情有没有办成”，智能体就会陷入一种危险的“执行幻觉”。

这正是物理 AI 如今面对的关键挑战：

- 模型越来越聪明，物理执行基础设施却依然碎片化；
- 机器人能够运动，但执行过程缺少统一的反馈与验收；
- Demo 可以跑起来，失败却往往难以定位、复现和积累；

- 换一个模型、仿真器或机器人，整条链路常常需要重新拼装。

这正是行业共识中"具身智能缺少一套操作系统级基础设施"的根源——模型层繁荣，执行层碎片化。



图：物理 AI 面对的关键挑战。

PhyAgentOS 的出发点，就是为这道缺口补上一层“操作系统”。

这一趋势也正在被更多前沿工作验证。近期 NVIDIA 相关团队提出的 CaP-X 与 ASPIRE，都把目光从“训练一个端到端策略”进一步扩展到“让智能体生成可执行控制程序、在执行反馈中发现问题，并把修正后的能力积累为可复用技能”。CaP-X 关注如何系统评测和改进 Code-as-Policy 机器人智能体，ASPIRE 则进一步探索通过执行轨迹、失败诊断和程序修复来自动发现机器人技能。

这说明，物理 AI 的关键问题已经不只是“模型能不能给出动作”，而是系统能不能支持模型、代码策略、仿真环境和真实机器人之间形成稳定闭环：任务如何下发，程序如何执行，失败如何定位，经验如何沉淀，技能如何复用。

PhyAgentOS 正是面向这一层问题，提供统一的运行、检查与追踪底座。

它不试图再造一个万能模型，也不替代 VLA、代码生成策略或世界模型。

如果说 VLA 像机器人的“条件反射”，把视觉、语言直接变成动作；代码即策略像一位会写操作步骤的工程师，把任务编译成程序；世界模型则像一座预演未来的模拟器，那么 **PhyAgentOS 更像管理这些能力的运行系统——决定何时调用、交给谁执行、如何检查、怎样接收反馈，以及失败后如何继续。**

它要做的，是在模型与真实执行对象之间，提供一套统一、透明、可审计的运行底座。

一句话理解 PhyAgentOS：把“让机器人做事”变成一套可管理的执行流程

大模型和 Agent 擅长理解意图、拆解目标，但真正进入物理世界时，任务不能只靠一句自然语言指令直接交给机器人。因为不同执行对象差异很大：游戏有游戏接口，仿真环境有仿真 API，机械臂、四足机器人和人形机器人又有各自的传感器、动作空间、控制频率和安全限制。没有统一的中间层，每接入一个新模型或新设备，都要重新写一套从“任务理解”到“动作执行”，再到“结果检查”的完整链路。

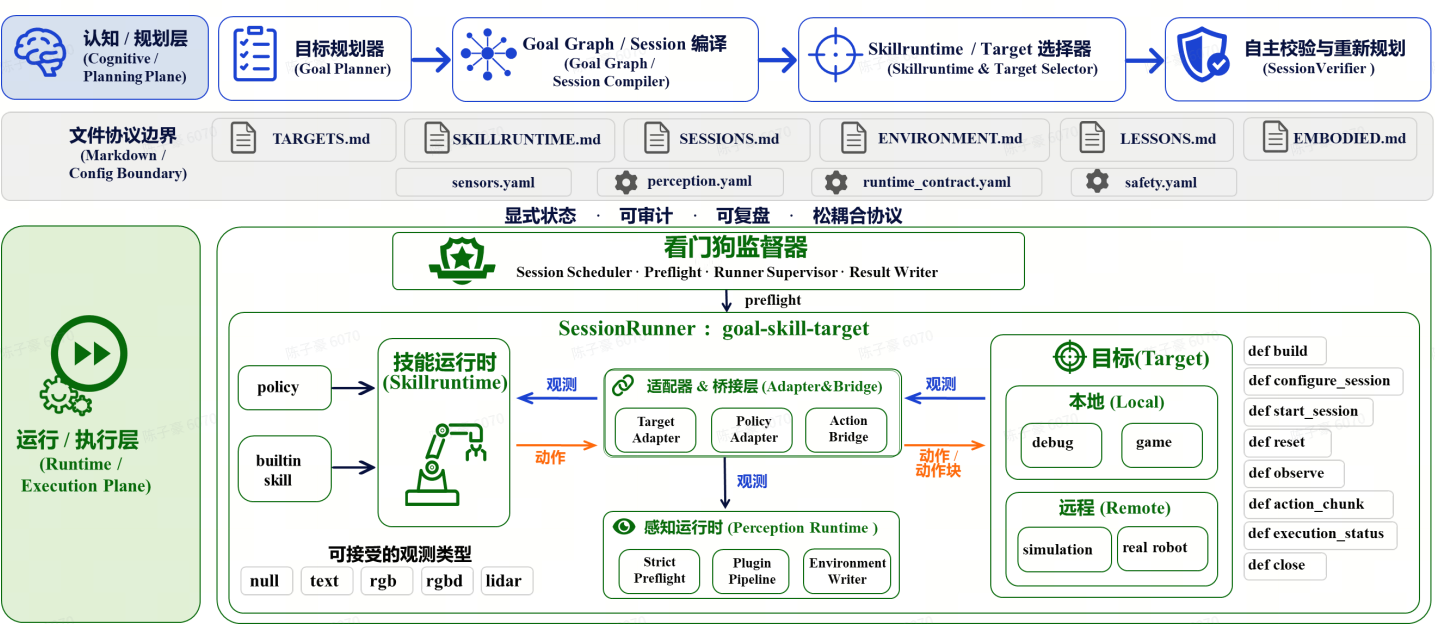
PhyAgentOS 要解决的，就是把这条链路标准化。

在 PhyAgentOS 中：

- **Target** 表示任务要实际作用的对象，可以是游戏、仿真环境，也可以是真实机器人；
- **Skill Runtime** 表示任务的执行方式，可以调用 VLA 等策略模型形成观察—动作闭环，也可以使用内置算法或受控工具完成任务；
- **Adapter 与 Bridge** 负责格式对齐，把不同平台的观察、动作和策略输出转换成统一、明确的协议；
- **Watchdog** 负责执行监督，包括任务排队、启动前检查、运行状态监控、超时处理和结果写回；
- **Session** 是一次完整任务的执行记录，描述任务交给谁、怎么执行、执行到哪一步、最终成功还是失败。

因此，Agent 不需要直接控制机器人，也不需要关心每个设备底层接口的差异。它只需提交一份结构化任务；PhyAgentOS 则负责检查这项任务是否能执行、选择合适的执行方式、连接对应的目标环境或机器人，并把执行过程和最终结果完整记录下来。

换句话说，PhyAgentOS 让物理智能体从“模型直接下命令”变成“任务经过系统调度后被安全执行”：模型负责理解和规划，Runtime 负责落地和监督，Target 负责真实执行，Session 负责留下可检查、可复现的全过程记录。



图：PhyAgentOS 将认知与规划交给 Agent 控制面，将实际执行交给 Session-Centered Runtime。

以 Session 为核心的物理执行单元

传统机器人系统常围绕驱动或单个动作组织：发出移动、抓取、停止指令，再由上层代码自行拼接。这种方式一旦进入长任务，状态、异常和资源释放就容易散落在不同模块里。

PhyAgentOS 把 **Session（会话）** 设为最小执行单元。

一次 Session 不只包含“做什么”，还会明确：

- 任务交给哪个 Target；
- 使用哪一种 Skill Runtime；
- 策略服务和远程执行端在哪里；
- 最长执行多久、最多运行多少步；
- 需要经过哪些 Adapter 与动作契约；
- 最终是成功、失败、超时，还是在执行前被拒绝。

Runtime 中的 WatchdogSupervisor 会先认领任务，再进行兼容性预检（Preflight）。只有目标、技能、传感器、动作格式、适配器关系和安全条件都满足要求，Session 才会真正启动。运行过程中，它还会持续监督心跳、超时、取消、策略服务、目标状态和 SafetyGuard。

之后，SessionRunner 负责完整生命周期，PolicySkillRuntime 或 BuiltinSkillRuntime 负责具体闭环，而 TargetSessionHandle 则把所有目标访问收束在一个受控入口中。

用大众熟悉的话来说：**过去是“把命令发出去”，现在是“把一件事情纳入全过程管理”。**

协议文件：认知与执行的桥梁

PhyAgentOS 没有把系统状态藏在某个难以查看的数据库里，而是用一组结构化 Markdown 文件构建 Agent 与 Runtime 的协议边界。

五份核心运行协议就像一套任何人都能打开阅读的“工作台账”：

- **TARGETS.md**——**设备名册**：有哪些执行目标，它们在哪里，支持什么能力；
- **SKILLRUNTIME.md**——**方法说明书**：有哪些执行方法，需要什么输入，会产生什么动作；
- **SESSIONS.md**——**任务工单**：这次要做什么，现在执行到哪一步，结果是什么；
- **ENVIRONMENT.md**——**现场地图**：当前目标、物体、场景关系和感知结果；
- **LESSONS.md**——**经验记录本**：预检、执行和验收过程中出现过哪些问题，下一次如何避免。

此外，**EMBODIED.md** 描述本体能力与限制，传感器、感知模型、动作契约和安全规则则通过外部 YAML 配置明确表达。它们共同构成完整的协议矩阵。

这种“状态显式化”带来一个朴素但重要的好处：**人能够看懂，Agent 能够读取，Runtime 能够验证，失败之后还能复盘。**

对于教学者，它是一张可以逐步讲解的系统流程图；对于研究者，它是一条可复现实验链；对于工程团队，它又是一套便于排障和审计的运行记录。

基于 Adapter 的兼容统一

不同平台之间最难迁移的，往往不是任务意图，而是数据格式。

一台机器人输出两路相机和 7 维关节状态，另一套仿真环境可能输出三路图像和 23 维状态；某个策略给出末端位姿增量，另一个策略给出关节位置序列。只要其中一个维度、坐标系或归一化方式没有对齐，系统就可能在运行时失败。

PhyAgentOS 使用三层明确的适配关系：

1. **TargetAdapter**：把不同目标的原始观察和动作格式转成 Runtime 能理解的形式；
2. **PolicyAdapter**：把 Runtime Observation 转成具体策略模型需要的输入，再把模型输出转回来；
3. **ActionBridge**：完成明确声明的动作转换与约束处理。

在执行前，Preflight 会检查这些契约是否真正匹配。它的原则很简单：**不兼容就明确拒绝，不靠偷偷截断、补零或猜测来“凑合运行”。**

这让问题从“机器人为什么突然不动了”，变成“第几个字段、哪一种动作表示不兼容”，大幅降低调试成本。

安全也因此不再只是一句写在文档里的提醒。PhyAgentOS 把安全分布在整条链路中：认知侧 Critic 检查任务意图，Runtime Preflight 检查执行契约，Target 侧 SafetyGuard 约束工作空间、速度与动作有效性，Operator Override 则为真实机器人保留人工接管权。每一层解决不同风险，共同避免让模型直接越过系统边界控制硬件。

让 Agent 作为任务检验的“标尺”

这是 PhyAgentOS 中最具代表性的能力之一：**SessionVerifier**。

Runtime 负责记录执行事实，但不越俎代庖判断自然语言任务是否真正完成。当一次执行结束后，系统可以把初始-最终 RGB 图像、原始任务、环境快照和历史记录整理成一份 Evidence Bundle，再交给 Agent 侧的 SessionVerifier 做语义验收。

它会给出三类结论：

- **success**：任务确实完成，Session 进入 succeeded；
- **failure**：执行结束但目标未达成，Session 进入 failed；
- **replan**：根据现场证据生成新的任务描述，原 Session 保留，新建一个待执行子 Session。

每次判断都会记录依据、经验和验证过程。即使任务失败，也不会只留下一句模糊的“运行异常”，而是形成可以检查、复盘和继续改进的证据链。

这意味着，PhyAgentOS 把“动作跑完了”升级为“任务被验收了”。

它也让“自进化”从口号变成闭环：成功经验进入长期记忆，失败原因沉淀到 [LESSONS.md](#)，需要修正的任务自动进入 replan，并生成新的执行 Session。系统在一次次真实交互中积累可复用经验，减少在相同环境里重复犯错。

从单次 Demo 到可复现评测：让实验也进入流水线

对于具身智能研究，最怕的不是一次失败，而是无法回答：失败发生在哪个任务、哪种初始状态、哪一个策略版本，又该如何复现？

PhyAgentOS 的 Benchmarking Skill 不另起一套执行系统，而是复用标准 Runtime：

1. 选择已经启用的 simulation Target 和 policy Skill Runtime；
2. 发现 Benchmark 任务与初始状态；
3. 批量生成标准 Session；
4. 由 Watchdog 和 SessionRunner 完成真实执行；
5. 汇总 episode.json、[LOG.md](#) 和 Session Result；
6. 计算成功率、每任务表现、步数、回报和错误分布，并保留复现配置。

PhyAgentOS 已经形成从 Game、Sim 到 Real 的多层验证体系：

- **General Game Agent**：在《我的世界》《星露谷物语》《饥荒》等可控环境中研究长期记忆、自主决策与自发进化；
- **LIBERO**：面向机器人操作的仿真 Benchmark；
- **CALVIN**：面向长程多步骤操作任务的机器人仿真 Benchmark；
- **RoboCasa365**：覆盖原子技能与复杂家庭活动的大规模移动操作 Benchmark；
- **Isaac Sim**：支持 PiperGo2 与 Merom 多机器人场景的远程 TargetWS；
- **OpenPI / pi0-family / X-VLA / RLDX-1 / WorldDreamer 等策略服务**：通过统一的 Policy Client 与 Adapter 接入不同 VLA 策略，实现即插即用、统一评测与执行闭环。

这让评测不再只是最后的一张结果表，而是一条从任务来源、执行过程到失败证据都可以追踪的实验流水线。

从 Game 到 Real：让智能先在可控世界成长，再回到真实物理世界

PhyAgentOS 的自进化路线并不是让机器人直接在高风险的真实环境中盲目试错，而是建立一条从 Game、Sim 到 Real 的渐进式迁移路径。

在游戏环境里，系统可以暂时剥离复杂动力学，把研究重点放在长期记忆、目标规划、资源管理和自主决策上。任务执行后，SessionVerifier 负责语义验收，成功策略沉淀为 Skill，失败案例进入 [LESSONS.md](#)，形成可以复现的正负样本。

进入仿真阶段后，TargetAdapter 与 PolicyAdapter 把这些策略映射到新的观察和动作空间，补上动力学、碰撞检测和控制延迟；进入真机阶段，再进一步引入硬件噪声、实时反馈和执行不确定性。

在这条路径上，Goal Planner 负责理解需求，Goal Graph 与 Session Compiler 负责拆解任务和维护依赖，Skill Runtime 与 Target Selector 为每个 Session 匹配合适的执行方法和对象；多机器人则通过共享状态协议空间动态分工、协同执行和积累经验。

智能层保持统一，物理假设逐步从“简化”回归“真实”。这让游戏不只是娱乐环境，仿真也不只是 Demo 场地，而是物理智能体安全学习、评测和进化的连续基础设施。

评估与进化 Showcase：让实验流水线与能力进化真实可见

Benchmarking Skill：行动、调度、执行、验收与失败恢复

一个机器人策略有没有进步，不能只看一次 Demo 跑没跑通。

真正重要的是：它在哪些任务上失败了？失败后系统能不能看见原因、留下证据，并尝试把任务继续完成？

PhyAgentOS 把这个过程放进了标准 Session 流水线。每一次评估不再只是运行一个脚本、得到一张表，而是从任务生成、环境初始化、策略执行、结果记录，到失败分析、Agent retry 和最终汇总，都被统一管理。

在 LIBERO 基准上，我们以 Pi0.5、X-VLA 等策略进行了自动化评估。结果显示，在四个 suite 共 2000 个 episode 中，Pi0.5 的首次执行成功率为 97.0%，经过 PhyAgentOS 的 Agent Retry 后，最终成功率提升到 97.8%；X-VLA 的首次执行成功率为 97.3%，最终提升到 98.6%。仅对 X-VLA，系统额外救回了 26 个原本失败的 episode——相当于在不修改任何模型代码的情况下，将失败数量从 54 个减少到 28 个。

在 CALVIN 长时序操作基准上，我们进一步验证了系统对连续任务的恢复能力。Pi0 的完整五步成功率由 38.9% 提升至 45.6%，Pi0.5 由 85.3% 提升至 89.4%，X-VLA 由 74.3% 提升至 75.7%，平均完成子任务数也均有所提升。这说明，PhyAgentOS 能够在不中断当前执行状态的情况下，对中途失败进行分析并恢复执行，从而提高长链任务的完成率。

在 RoboCasa365 家庭操作基准上，我们验证了系统在复杂家庭场景中的恢复效果。Pi0.5 的总体成功率由 17.6% 提升至 26.8%，RLDX-1 由 35.6% 提升至 42.8%，WorldDreamer 由 34.0% 提升至 42.4%。系统分别额外救回 23、18 和 21 个失败 episode，说明失败恢复机制同样能够有效提升复杂家庭操作任务的最终完成率。

综合三个 Benchmark 可以看到，PhyAgentOS 并不修改策略模型或 Benchmark 定义，而是在统一的 Session Runtime 中，将任务调度、策略执行、结果验收、失败分析和 Agent Retry组织成完整的执行流程。First Attempt 始终代表策略自身的原始能力，Final Outcome 则代表系统经过失败分析与恢复后的最终执行能力。Benchmark 不再只是一次成功率统计，而成为一条能够记录失败、恢复执行并持续积累经验完整评测流水线。

这背后的关键，不是简单“多试一次”，而是让失败变得可检查、可复盘、可继续。

当一个 episode 失败时，PhyAgentOS 会记录任务描述、初始观测、最终观测、运行状态和失败信息，并交给 Agent 判断是否需要重新规划。成功的任务进入结果统计；失败的任务进入证据链、重试链和经验库。

因此，PhyAgentOS 严格区分两类结果：**first attempt** 代表策略本身在标准评估下的原始表现；**final outcome** 代表系统在失败分析和恢复介入后的最终完成能力。两者不会混在一起计算，也不会把多次重试错误地当成新的 episode。

这让 benchmark 不再只是论文最后的一张结果表，而是一条真实可见的进化流水线：模型负责行动，Runtime 负责调度，Target 负责执行，Verifier 负责验收，Agent 负责在失败后尝试修正。

PhyAgentOS 想证明的不是“模型永远不会失败”，而是：当失败发生时，系统能够看见它、记录它、理解它，并把它变成下一次执行更可靠的经验。

Model	Setting	Spatial	Object	Goal	LIBERO-10	Overall
OpenVLA	First	84.4%	86.4%	74.6%	52.6%	74.5%
	Final	85.8%	87.6%	75.2%	53.4%	75.5%
	Gain	+1.4	+1.2	+0.6	+0.8	+1.0
π_0	First	96.8%	98.2%	93.4%	82.6%	92.8%
	Final	97.6%	98.2%	94.2%	82.6%	93.2%
	Gain	+0.8	+0.0	+0.8	+0.0	+0.4
$\pi_{0.5}$	First	99.4%	98.8%	97.2%	92.8%	97.0%
	Final	99.6%	98.8%	98.0%	94.6%	97.8%
	Gain	+0.2	+0.0	+0.8	+1.8	+0.8
X-VLA	First	97.2%	97.4%	98.0%	96.6%	97.3%
	Final	99.0%	97.8%	99.0%	98.6%	98.6%
	Gain	+1.8	+0.4	+1.0	+2.0	+1.3

图：Pi0.5、X-VLA 等策略在 LIBERO 基准上的基础自动评估与 PhyAgentOS 介入性能对比。

Model	Setting	1/5	2/5	3/5	4/5	5/5	Avg. Len.
X-VLA	First	96.8%	92.0%	86.2%	81.7%	74.3%	4.310
	Final	97.0%	92.0%	86.3%	81.9%	75.7%	4.329
	Gain	+0.2	+0.0	+0.1	+0.2	+1.4	+0.019
π_0	First	86.5%	74.0%	62.2%	50.9%	38.9%	3.125
	Final	86.8%	74.7%	64.1%	53.7%	45.6%	3.249
	Gain	+0.3	+0.7	+1.9	+2.8	+6.7	+0.124
$\pi_{0.5}$	First	99.7%	98.0%	94.5%	91.5%	85.3%	4.690
	Final	99.7%	98.5%	95.2%	92.5%	89.4%	4.753
	Gain	+0.0	+0.5	+0.7	+1.0	+4.1	+0.063

图：Pi0、Pi0.5、X-VLA 策略在 CALVIN 基准上的基础自动评估与 PhyAgentOS 介入性能对比。

Model	Setting	Atomic	Composite	Overall	Rescued
$\pi_{0.5}$	First	41.1%	4.4%	17.6%	23
	Final	56.7%	10.0%	26.8%	
	Gain	+15.6	+5.6	+9.2	
RLDX-1	First	70.0%	16.2%	35.6%	18
	Final	75.6%	24.4%	42.8%	
	Gain	+5.6	+8.1	+7.2	
WorldDreamer	First	66.7%	15.6%	34.0%	21
	Final	73.3%	25.0%	42.4%	
	Gain	+6.7	+9.4	+8.4	

图：Pi0.5、RLDX-1、WorldDreamer 策略在 RoboCasa365 基准上的基础自动评估与 PhyAgentOS 介入性能对比。

Game Agent：在开放世界沙盒中验证长期闭环与自进化

游戏环境既保留了资源消耗、时间压力、状态衰减等接近物理世界的约束，又具备低成本、高并发、易复现的实验优势，因此成为 PhyAgentOS 验证智能体闭环能力的理想场地。基于统一的 Runtime 与 Memory 接口，PhyAgentOS 能够让 Agent 在跨 episode 的持续运行中积累长期记忆、沉淀可复用技能，并逐步实现自我进化。

饥荒：突破生存瓶颈。 在《饥荒》秋季 Day 0 白天场景（10 episodes）中，DeepSeek v4 Flash 裸跑平均存活 1.02 天，90% 死于第一夜 Charlie；接入 PhyAgentOS 后存活天数提升至 2.10 天，Day 3 生存率从 0% 提升至 30%。裸跑 Agent 虽然“知道”要生火，但缺乏执行时序与状态反馈的闭环管理。

PhyAgentOS 通过 Runtime Contract 和动作追踪，使"天黑前生火"被可靠执行。同时，PhyAgentOS 模式下平均 Health、Hunger 和 Sanity 更低——这并非性能退化，而是因存活时间翻倍，数据覆盖了更长的消耗周期。

星露谷 Lite-100：跨 Episode 记忆与技能进化。 在 StarDojo Lite-100 多任务 benchmark 中，PhyAgentOS 在 DeepSeek v4 Flash 上取得总分 **22.0%**，超过 SPIKE (Qwen3.5-397B) 的 18.0% 与 SPIKE (GPT5.4) 的 20.3%。分项来看，PhyAgentOS 在 Crafting 上达到 **50.0%**（此前最高 baseline 为 19.0%），在 Combat 上达到 **16.7%**（SPIKE 为 8.3%），验证了操作系统层对复杂多步任务和战斗闭环的显著增益。Farming 与 Exploration 分别为 28.6% 与 17.9%，仅次于参数量更大的 SPIKE。这一结果说明：通过 Memory 接口将跨 episode 的种植周期、工具配方和 NPC 交互历史写入结构化记忆，Agent 能够将失败修复后的策略沉淀为可复用 Skill，实现持续自进化。

向真实机器人迁移。 游戏验证的核心经验是：物理智能体的瓶颈不是"模型能不能想出正确动作"，而是"系统能不能确保动作在正确时机被正确执行，并拿到真实反馈"。《饥荒》中的"生火动作晚于天黑"与机械臂中的"抓取指令在物体滑落后才到达"本质上是同一类问题。PhyAgentOS 通过同一套 Session 协议和追踪机制，将游戏中的长期记忆与自进化能力直接迁移到 AgileX PIPER、Franka Research 3 和 Unitree Go2 等真实平台。

Model	Farming ↑	Crafting ↑	Exploration ↑	Combat ↑	Social ↑	Total ↑
ReAct-like	14.3%	9.5%	4.8%	0.0%	4.0%	6.7%
Reflexion-like	17.5%	11.9%	4.8%	2.8%	6.7%	8.7%
Voyager-like	14.3%	9.5%	6.0%	2.8%	4.0%	7.3%
CRADLE	25.4%	16.7%	10.7%	2.8%	8.0%	13.0%
StarDojo	20.6%	19.0%	9.5%	5.6%	8.0%	12.3%
SPIKE (Qwen3.5-397B)	34.9%	23.8%	13.1%	8.3%	10.7%	18.0%
SPIKE (Gemini-3.1-pro)	-	-	-	-	-	17.7%
SPIKE (GPT5.4)	-	-	-	-	-	20.3%
PhyAgentOS	28.6%	50.0%	17.9%	16.7%	8.0%	22.0%

图：在 StarDojo Lite-100 上，PhyAgentOS 与 SPIKE 的各分类结果对比，SPIKE 的数值来自该论文报告的平均值。结果表明我们的PhyAgentOS仅依赖DeepSeek v4 flash即可超过GPT5.4的性能。

Metric	Raw LLM	+ PhyAgentOS	Change
Survival Days	1.02 ± 0.08	2.10 ± 0.88	+106%
Day 3 Survival Rate	0%	30%	+30 pp
Death by Charlie (darkness)	90%	80%	-10 pp
Death by Monster	10%	10%	—
Death by Starvation	0%	10%	+10 pp
Avg Health (per 10 s)	0.870	0.832	-0.038
Avg Hunger (per 10 s)	0.827	0.650	-0.177
Avg Sanity (per 10 s)	0.962	0.880	-0.082

图：DeepSeek v4 Flash 在《饥荒》秋季第 0 天、白天阶段（10 轮 episode）上的表现。
PhyAgentOS 使 Agent 存活时间翻倍，并突破了第 3 天的生存阈值。

真机与多本体 Showcase：让统一协议落到不同形态本体上

PhyAgentOS 已在多种主流机器人平台以及游戏、仿真平台上验证协议化接入与任务执行：

真实机器人：

- **Franka Research 3**：视觉推理、抓取与 ReKep 部署验证；
- **AgileX PIPER**：ReKep、SAM3 自然语言抓取；
- **Dobot Nova 2**：基于关系关键点约束的精细操作；
- **Unitree Go2**：移动能力与语义导航；
- **XLeRobot**：双臂协调与抓取；
- **Unitree G1 / 零次方人形机器人**：语音交互与动作执行。

仿真平台：

- **MuJoCo**：支持 **LIBERO**、**RoboCasa365** 等机器人操作 Benchmark，实现高效策略训练与快速评测。
- **PyBullet**：支持 **CALVIN** 长程操作 Benchmark，用于连续多步骤任务验证与恢复能力评估。
- **Isaac Sim**：支持高保真复杂场景生成，用于多机器人部署与调度。

更重要的是，这些本体并不是一组彼此孤立的 Demo。它们通过 **TARGETS.md** 暴露能力，通过 Session 接收任务，通过共享的 **ENVIRONMENT.md** 理解状态，并在 Goal Graph 中维护前后依赖。一个复杂任务可以被拆成导航、观察、抓取、交接等多个 Session，再分配给最适合的本体完成。

这使 PhyAgentOS 从“让一台机器人动起来”，进一步走向“让一群不同形态的机器人围绕同一目标协同工作”。

PhyAgentOS 适合谁？

对研究者：把时间留给真正的问题

研究者可以把新策略、新感知方法或新仿真环境接入统一 Session 协议，不必为每项实验重新编写调度、状态管理和结果落盘逻辑。成功与失败都留下结构化证据，便于公平比较与论文复现。

对高校教师和学生：让“智能体闭环”变得看得见

从用户请求、任务工单、兼容性检查，到观察—推理—动作闭环和最终验收，关键状态都能直接查看。学生不只看到机器人“动了”，还能理解一个物理智能体系统为什么这样设计、在哪一步失败。

对模型团队：让策略不再被单一平台绑住

模型团队可以通过 Policy Client 与 PolicyAdapter 接入现有 Runtime，在 LIBERO、BEHAVIOR-1K 和 Isaac Sim 等不同目标上验证输入输出契约、闭环行为和评测结果。

对机器人与自动化团队：先建立可控边界，再走向真实部署

PhyAgentOS 为真实机器人接入提供了清晰路径：Target Runtime、Adapter、Runtime Contract、受控 Tool 和本地安全机制各司其职。系统已经在 AgileX PIPER、Dobot Nova 2、Franka Research 3、Unitree Go2、XLeRobot、Unitree G1/零次方等平台完成验证或适配，覆盖机械臂、四足、双臂与人形机器人。

快速开始：先让第一条 Session 跑起来

代码块

```
1 git clone https://github.com/PhyAgentOS/PhyAgentOS.git
2 cd PhyAgentOS
3 python -m pip install -e .
4 paos onboard
5 paos agent
```

PhyAgentOS 提供不依赖真实硬件的 Dummy Runtime，也支持连接游戏环境、LIBERO、BEHAVIOR-1K、Isaac Sim、OpenPI 与真实机器人 Target。用户可以从第一条 Session 开始，逐步扩展到策略评测、长程任务和多本体协同。

结语：物理 AI 的下一步，不只是更强的模型

从 VLA 到代码即策略，从世界模型到多模态推理，我们正在快速获得越来越聪明的“大脑”。但要让这些大脑真正进入现实世界，仅有模型还不够。

物理智能体还需要一套能够管理任务、检查契约、监督执行、接收反馈、验收结果并积累经验的系统基础设施。

这正是 PhyAgentOS 想回答的问题：

当 AI 开始触碰真实世界，我们不仅要让它会想、会做，更要让每一次行动都有边界、每一次结果都有证据、每一次失败都能成为下一次进步的起点。

物理 AI 的“觉醒时刻”，或许才刚刚开始。